

打造全端電影推薦系統

來源：<https://codelabs.developers.google.com/tfrecommenders-flutter?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將建構完整堆疊推薦系統。您將使用 TensorFlow 建議工具訓練 2 個推薦模型，並使用 TensorFlow Serving 做為後端進行部署。您也會建構跨平台 Flutter 應用程式做為前端。

目錄

1. 事前準備
2. 設定 Flutter 開發環境
3. 做好準備
4. 下載專案的依附元件
5. 步驟 0：執行範例應用程式
6. 步驟 1：為推薦引擎建立擷取和排名模型
7. 步驟 2：建立推薦引擎後端
8. 步驟 3：為 Android 和 iOS 建立 Flutter 應用程式
9. 步驟 4：在桌面平台執行 Flutter 應用程式
10. 步驟 5：在網頁平台上執行 Flutter 應用程式
11. 恭喜

1. 事前準備

推薦引擎 (又稱推薦系統) 是機器學習非常重要的應用，可推薦電影或餐廳，也能精選有趣的影片。推薦內容可協助您從大量候選內容中，找出使用者感興趣的內容。舉例來說，Google Play 商店提供數百萬個應用程式供您安裝，而 YouTube 則提供數十億部影片供您觀看。每天都會新增更多應用程式和影片。

在本程式碼研究室中，您將瞭解如何使用下列項目建構全端推薦系統：

- 使用 TensorFlow Recommenders 訓練電影推薦的擷取和排名模型
- 使用 TensorFlow Serving 提供模型
- 使用 Flutter 建立跨平台應用程式，顯示推薦電影

必要條件

- 具備使用 Dart 進行 Flutter 開發的基本知識
- 具備 TensorFlow 機器學習基本知識，例如訓練與部署的差異
- 熟悉推薦系統的基本概念
- Python、終端機和 Docker 的基本知識

課程內容

- 如何使用 TensorFlow Recommenders 訓練擷取和排名模型
- 如何使用 TensorFlow Serving 提供訓練好的推薦模型
- 如何建構跨平台的 Flutter 應用程式，顯示建議項目

軟硬體需求

- [Flutter SDK](#)
 - 為 Flutter 設定 [Android](#) 和 [iOS](#)
 - Flutter 的[電腦](#)設定
 - Flutter 的[網頁](#)設定
 - 設定 [Visual Studio Code \(VS Code\)](#)，以便使用 Flutter 和 Dart
 - [Docker](#)
 - [Bash](#)
 - Python 3.7 以上版本
 - 存取 [Colab](#)
-

步驟 2 · 約 10 分鐘

2. 設定 Flutter 開發環境

如要進行 Flutter 開發，您需要兩項軟體才能完成本程式碼研究室：[Flutter SDK](#) 和[編輯器](#)。

您可以使用下列任一裝置執行程式碼研究室的前端：

- [iOS 模擬器](#) (需要安裝 Xcode 工具)。
- [Android Emulator](#) (需在 Android Studio 中設定)。
- 瀏覽器 (偵錯時必須使用 Chrome)。
- 以 [Windows](#)、[Linux](#) 或 [macOS](#) 電腦版應用程式的形式。您必須在要部署的平台上進行開發。因此，如要開發 Windows 桌面應用程式，您必須在 Windows 上開發，才能存取適當的建構鏈。如需作業系統專屬需求，請參閱 docs.flutter.dev/desktop。

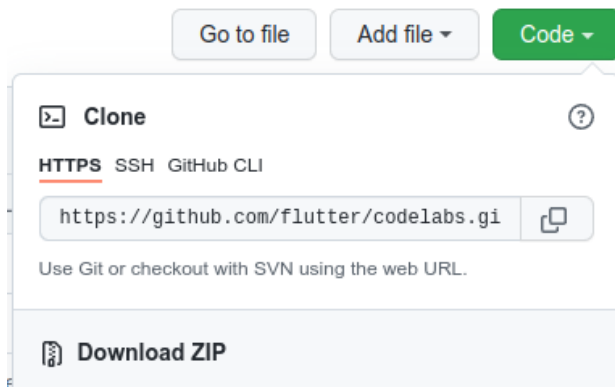
後端需求：

- Linux 電腦或搭載 Intel 處理器的 Mac。
-

3. 做好準備

如要下載本程式碼研究室的程式碼，請按照下列步驟操作：

1. 前往本程式碼研究室的 [GitHub 存放區](#)。
2. 依序點選「Code」>「Download zip」，下載這個程式碼研究室的所有程式碼。



3. 將下載的 ZIP 檔案解壓縮，解壓縮後會產生 `codelabs-main` 根資料夾，內含所有需要的資源。

在本程式碼研究室中，您只需要存放區中 `tfrs-flutter/` 子目錄的檔案，其中包含多個資料夾：

- `step0` 至 `step5` 資料夾包含範例程式碼，您將在本程式碼研究室的每個步驟中，以這些程式碼為基礎進行建構。
 - `finished` 資料夾包含完成的程式碼，適用於完成的範例應用程式。
 - 每個資料夾都包含 `backend` 子資料夾 (內含建議引擎後端程式碼) 和 `frontend` 子資料夾 (內含 Flutter 前端程式碼)
-

步驟 4 · 約 3 分鐘

4. 下載專案的依附元件

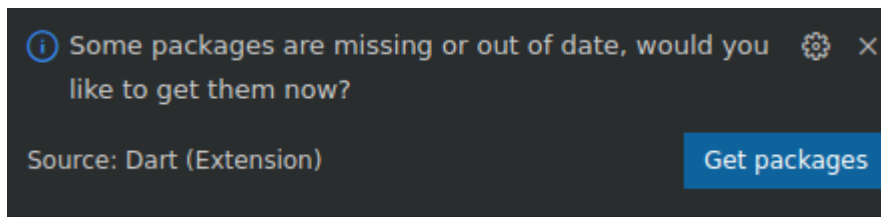
後端

我們將使用 [Flask](#) 建立後端。開啟終端機並執行下列指令：

```
pip install Flask flask-cors requests numpy
```

前端

1. 在 VS Code 中，依序點選「File」>「Open folder」，然後選取先前下載的原始碼中的 `step0` 資料夾。
2. 開啟 `step0/frontend/lib/main.dart` 檔案如果看到 VS Code 對話方塊，提示您下載範例應用程式的必要套件，請按一下「Get packages」(取得套件)。
3. 如果沒有看到這個對話方塊，請開啟終端機，然後在 `step0/frontend` 資料夾中執行 `flutter pub get` 指令。

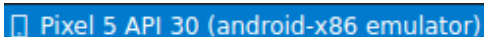


步驟 5 · 約 3 分鐘

5. 步驟 0：執行範例應用程式

1. 在 VS Code 中開啟 `step0/frontend/lib/main.dart` 檔案，確認 Android Emulator 或 iOS 模擬器已正確設定，並顯示在狀態列中。

舉例來說，在 Android 模擬器中使用 Pixel 5 時，畫面會顯示以下內容：



以下是使用 iOS 模擬器搭配 iPhone 13 時的畫面：



2. 按一下「Start debugging」

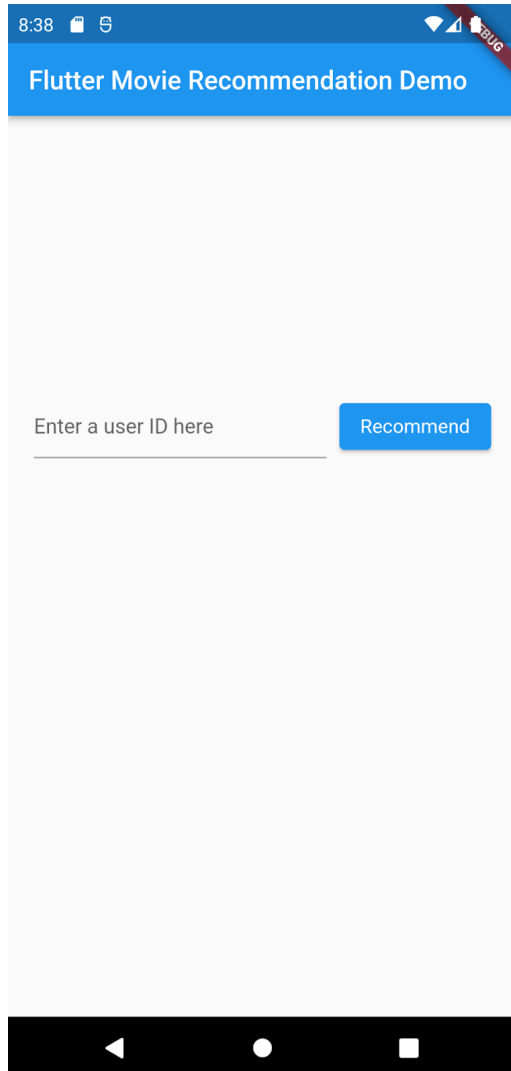


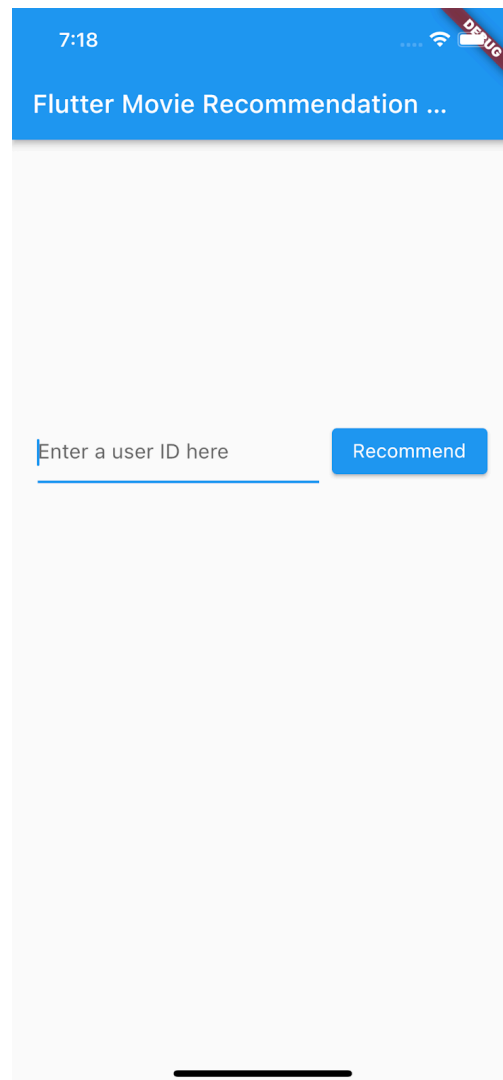
。

注意：本程式碼研究室假設您在本機電腦上部署建議模型。如果要在其他遠端機器上部署模型，請將程式碼中的 `10.0.2.2` 或 `127.0.0.1` IP 位址變更為執行 TensorFlow Serving 的遠端機器 IP 位址。

執行並探索應用程式

應用程式應會在 Android 模擬器或 iOS 模擬器上啟動。使用者介面相當簡單明瞭。使用者可以在文字欄位中輸入使用者 ID。Flutter 應用程式會將查詢要求傳送至後端，後端會執行 2 個推薦模型，並傳回電影推薦的排名清單。前端收到回應後，就會在 UI 中顯示結果。



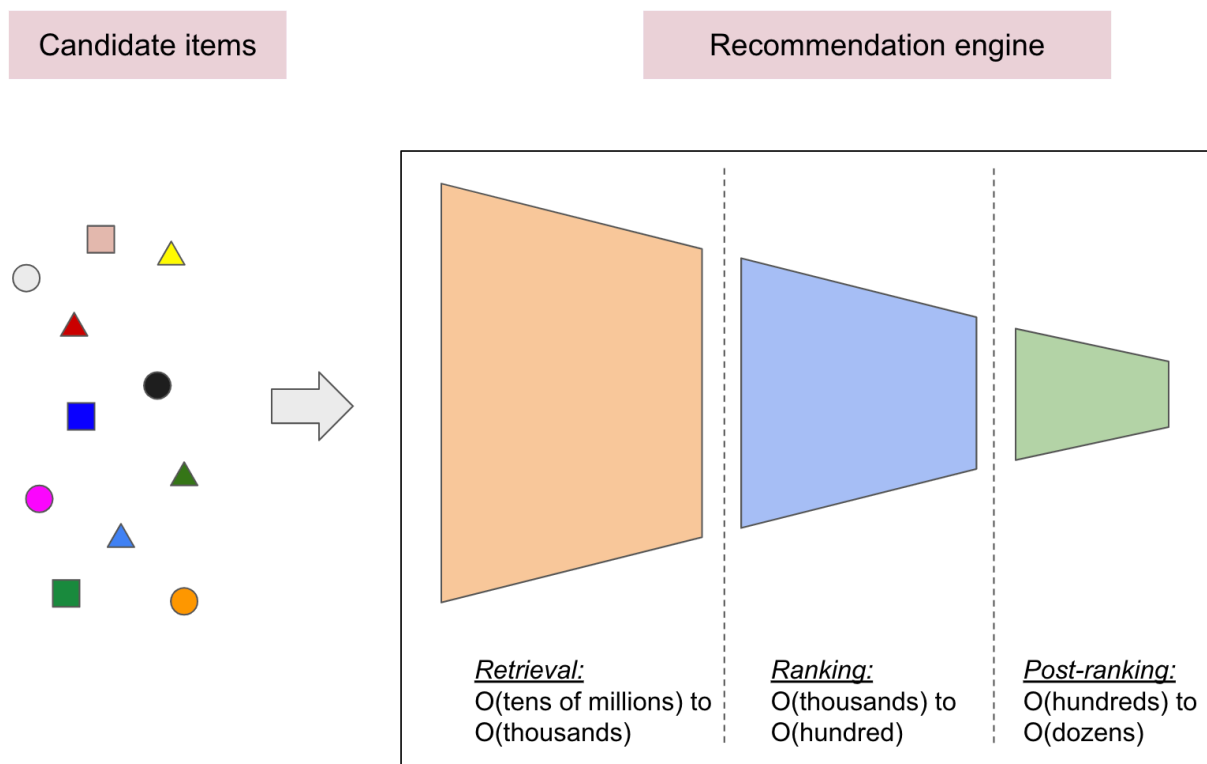


如果您現在點選「Recommend」，系統不會有任何反應，因為應用程式還無法與後端通訊。

6. 步驟 1：為推薦引擎建立擷取和排名模型

真實世界的推薦引擎通常由多個階段組成：

1. 擷取階段負責從所有可能的候選項目中，選取最初的數百個候選項目。這個模型的主要目標，是有效排除使用者不感興趣的所有候選人。由於擷取模型可能要處理數百萬個候選項目，因此必須具備運算效率。
2. 排名階段會採用擷取模型的輸出內容並微調，選取最合適的少量建議。這項工作的目標是將使用者可能感興趣的項目縮減為候選清單，數量約為數百個。
3. 排名後階段有助於確保多元性、新鮮度和公平性，並將候選項目重新整理為數十個實用建議。



在本程式碼研究室中，您會使用熱門的 MovieLens 資料集，訓練擷取模型和排序模型。您可以透過 Colab 開啟下列訓練程式碼，然後按照說明操作：

- [MovieLens 擷取模型](#)
- [MovieLens 排名模型](#)

注意：訓練程式碼改編自 TensorFlow Recommenders 的[基本擷取](#)和[基本排序](#)教學課程。詳情請參閱教學課程。此外，您也可以參考[這部影片](#)，瞭解如何建構基本擷取模型，以及[這部影片](#)，瞭解如何建構基本排序模型。

7. 步驟 2：建立推薦引擎後端

注意：下列操作說明僅適用於 Linux 電腦或搭載 Intel 晶片的 Mac。您或許可以將指令改為適用於 Windows。詳情請參閱 Docker 說明文件。由於 TensorFlow Serving 在 2024 年 1 月前不相容，因此無法在 Apple Silicon 架構的 Mac 上運作。

現在您已訓練擷取和排序模型，可以部署模型並建立後端。

附註：為方便起見，預先訓練的 SavedModel 模型會放在 `step2/backend/retrieval/exported-retrieval/` 和 `step2/backend/ranking/exported-ranking/` 資料夾中。

啟動 TensorFlow Serving

由於您需要同時使用擷取和排名模型來產生建議電影清單，因此請使用 TensorFlow Serving 同時部署這兩個模型。

- 在終端機中，前往電腦上的 `step2/backend` 資料夾，然後使用 Docker 啟動 TensorFlow Serving：

```
docker run -t --rm -p 8501:8501 -p 8500:8500 -v "$(pwd)/:/models/" tensorflow/serving
--model_config_file=/models/models.config
```

注意：`models.config` 檔案會指定這兩個模型在 TensorFlow Serving 中的部署方式。如果您自行從 Colab 訓練及下載模型，請務必更新 `models.config` 檔案中的模型路徑。

Docker 會先自動下載 TensorFlow Serving 映像檔，這需要一分鐘。TensorFlow Serving 隨即會啟動。記錄應如下列程式碼片段所示：

```
2022-04-24 09:32:06.461702: I tensorflow_serving/model_servers/server_core.cc:465]
Adding/updating models.
2022-04-24 09:32:06.461843: I tensorflow_serving/model_servers/server_core.cc:591]
(Re-)adding model: retrieval
2022-04-24 09:32:06.461907: I tensorflow_serving/model_servers/server_core.cc:591]
(Re-)adding model: ranking
2022-04-24 09:32:06.576920: I tensorflow_serving/core/basic_manager.cc:740]
Successfully reserved resources to load servable {name: retrieval version: 123}
2022-04-24 09:32:06.576993: I tensorflow_serving/core/loader_harness.cc:66] Approving
load for servable version {name: retrieval version: 123}
2022-04-24 09:32:06.577011: I tensorflow_serving/core/loader_harness.cc:74] Loading
servable version {name: retrieval version: 123}
2022-04-24 09:32:06.577848: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:38] Reading SavedModel
from: /models/retrieval/exported-retrieval/123
2022-04-24 09:32:06.583809: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:90] Reading meta graph
with tags { serve }
2022-04-24 09:32:06.583879: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:132] Reading SavedModel
debug info (if present) from: /models/retrieval/exported-retrieval/123
2022-04-24 09:32:06.584970: I
external/org_tensorflow/tensorflow/core/platform/cpu_feature_guard.cc:142] This
TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to
use the following CPU instructions in performance-critical operations: AVX2 FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler
flags.
2022-04-24 09:32:06.629900: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:206] Restoring SavedModel
bundle.
2022-04-24 09:32:06.634662: I
external/org_tensorflow/tensorflow/core/platform/profile_utils/cpu_utils.cc:114] CPU
Frequency: 2800000000 Hz
2022-04-24 09:32:06.672534: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:190] Running
initialization op on SavedModel bundle at path: /models/retrieval/exported-
retrieval/123
2022-04-24 09:32:06.673629: I tensorflow_serving/core/basic_manager.cc:740]
Successfully reserved resources to load servable {name: ranking version: 123}
2022-04-24 09:32:06.673765: I tensorflow_serving/core/loader_harness.cc:66] Approving
load for servable version {name: ranking version: 123}
2022-04-24 09:32:06.673786: I tensorflow_serving/core/loader_harness.cc:74] Loading
servable version {name: ranking version: 123}
2022-04-24 09:32:06.674731: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:38] Reading SavedModel
from: /models/ranking/exported-ranking/123
2022-04-24 09:32:06.683557: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:90] Reading meta graph
```

```
with tags { serve }
2022-04-24 09:32:06.683601: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:132] Reading SavedModel
debug info (if present) from: /models/ranking/exported-ranking/123
2022-04-24 09:32:06.688665: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:277] SavedModel load for
tags { serve }; Status: success: OK. Took 110815 microseconds.
2022-04-24 09:32:06.690019: I
tensorflow_serving/servables/tensorflow/saved_model_warmup_util.cc:59] No warmup data
file found at /models/retrieval/exported-
retrieval/123/assets.extra/tf_serving_warmup_requests
2022-04-24 09:32:06.693025: I tensorflow_serving/core/loader_harness.cc:87]
Successfully loaded servable version {name: retrieval version: 123}
2022-04-24 09:32:06.702594: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:206] Restoring SavedModel
bundle.
2022-04-24 09:32:06.745361: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:190] Running
initialization op on SavedModel bundle at path: /models/ranking/exported-ranking/123
2022-04-24 09:32:06.772363: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:277] SavedModel load for
tags { serve }; Status: success: OK. Took 97633 microseconds.
2022-04-24 09:32:06.774853: I
tensorflow_serving/servables/tensorflow/saved_model_warmup_util.cc:59] No warmup data
file found at /models/ranking/exported-
ranking/123/assets.extra/tf_serving_warmup_requests
2022-04-24 09:32:06.777706: I tensorflow_serving/core/loader_harness.cc:87]
Successfully loaded servable version {name: ranking version: 123}
2022-04-24 09:32:06.778969: I tensorflow_serving/model_servers/server_core.cc:486]
Finished adding/updating models
2022-04-24 09:32:06.779030: I tensorflow_serving/model_servers/server.cc:367]
Profiler service is enabled
2022-04-24 09:32:06.784217: I tensorflow_serving/model_servers/server.cc:393] Running
gRPC ModelServer at 0.0.0.0:8500 ...
[warn] getaddrinfo: address family for nodename not supported
2022-04-24 09:32:06.785748: I tensorflow_serving/model_servers/server.cc:414]
Exporting HTTP/REST API at:localhost:8501 ...
[evhttp_server.cc : 245] NET_LOG: Entering the event loop ...
```

注意：啟動 Tensorflow Serving Docker 映像檔後，您可以前往

http://localhost:8501/v1/models/<MODEL_NAME>/metadata，檢查輸入和輸出張量的詳細資料。在本程式碼研究室中，請將 <MODEL_NAME> 替換為 `retrieval` 或 `ranking`。

建立新端點

由於 TensorFlow Serving 不支援「串連」多個循序模型，因此您需要建立新服務，連結擷取和排序模型。

- 將這段程式碼新增至 `step2/backend/recommendations.py` 檔案中的 `get_recommendations()` 函式：

```
user_id = request.get_json()["user_id"]
retrieval_request = json.dumps({"instances": [user_id]})
retrieval_response = requests.post(RETRIEVAL_URL, data=retrieval_request)
movie_candidates = retrieval_response.json()["predictions"][0]["output_2"]

ranking_queries = [
    {"user_id": u, "movie_title": m}
    for (u, m) in zip([user_id] * NUM_OF_CANDIDATES, movie_candidates)
]
ranking_request = json.dumps({"instances": ranking_queries})
ranking_response = requests.post(RANKING_URL, data=ranking_request)
movies_scores = list(np.squeeze(ranking_response.json()["predictions"]))
ranked_movies = [
    m[1] for m in sorted(list(zip(movies_scores, movie_candidates)), reverse=True)
]

return make_response(jsonify({"movies": ranked_movies}), 200)
```

啟動 Flask 服務

現在可以啟動 Flask 服務。

- 在終端機中，前往 `step2/backend/` 資料夾並執行下列指令：

```
FLASK_APP=recommender.py FLASK_ENV=development flask run
```

Flask 會在 `http://localhost:5000/recommend` 建立新端點。您應該會看到如下的記錄：

```
* Serving Flask app 'recommender.py' (lazy loading)
* Environment: development
* Debug mode: on
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 705-382-264
127.0.0.1 - - [25/Apr/2022 19:44:47] "POST /recommend HTTP/1.1" 200 -
```

您可以向端點傳送範例要求，確認端點是否正常運作：

```
curl -X POST -H "Content-Type: application/json" -d '{"user_id": "42"}'
http://localhost:5000/recommend
```

端點會傳回使用者 `42` 的建議電影清單：

```
{
  "movies": [
    "While You Were Sleeping (1995)",
    "Preacher's Wife, The (1996)",
    "Michael (1996)",
    "Lion King, The (1994)",
    "Father of the Bride Part II (1995)",
    "Sleepless in Seattle (1993)",
    "101 Dalmatians (1996)",
    "Bridges of Madison County, The (1995)",
    "Rudy (1993)",
    "Jack (1996)"
  ]
}
```

大功告成！您已成功建構後端，根據使用者 ID 推薦電影。

注意：在本程式碼研究室中，您會將擷取和排序模型都部署到單一 TensorFlow Serving 執行個體。在實務上，通常會將這些工作負載部署到不同的叢集，以便更妥善地管理實際工作環境工作負載。

8. 步驟 3：為 Android 和 iOS 建立 Flutter 應用程式

後端已準備就緒，現在可以開始向該服務傳送要求，從 Flutter 應用程式查詢電影推薦。

前端應用程式相當簡單。這個畫面只有一個 `TextField`，可接收使用者 ID，並將要求 (在 `recommend()` 函式中) 傳送至您剛建構的後端。收到回應後，應用程式 UI 會在 `ListView` 中顯示建議的電影。

- 將這段程式碼新增至 `step3/frontend/lib/main.dart` 檔案中的 `recommend()` 函式：

```
final response = await http.post(
  Uri.parse('http://' + _server + ':5000/recommend'),
  headers: <String, String>{
    'Content-Type': 'application/json',
  },
  body: jsonEncode(<String, String>{
    'user_id': _userIDController.text,
  }),
);
```

應用程式收到後端的回應後，您會更新 UI，顯示指定使用者的建議電影清單。

- 在上述程式碼下方新增下列程式碼：

```
if (response.statusCode == 200) {
  return List<String>.from(jsonDecode(response.body)['movies']);
} else {
  throw Exception('Error response');
}
```

開始執行

- 按一下「Start debugging」圖示



，然後等待應用程式載入。

- 輸入使用者 ID (即 42)，然後選取「建議」。

Flutter Movie Recommendation Demo

42

Recommend



While You Were Sleeping (1995)



Preacher's Wife, The (1996)



Michael (1996)



Lion King, The (1994)



Father of the Bride Part II (1995)



Sleepless in Seattle (1993)



101 Dalmatians (1996)



Bridges of Madison County, The (1995)



Rudy (1993)



Jack (1996)

7:19



Flutter Movie Recommendation ...

42

Recommend



While You Were Sleeping (1995)



Preacher's Wife, The (1996)



Michael (1996)



Lion King, The (1994)



Father of the Bride Part II (1995)



Sleepless in Seattle (1993)



101 Dalmatians (1996)



Bridges of Madison County, The (1995)



Rudy (1993)



Jack (1996)

9. 步驟 4：在桌面平台執行 Flutter 應用程式

除了 Android 和 iOS，Flutter 也支援 Linux、Mac 和 Windows 等電腦平台。

Linux

1. 確認目標裝置在 VSCode 的狀態列中設為

Linux (linux-x64)

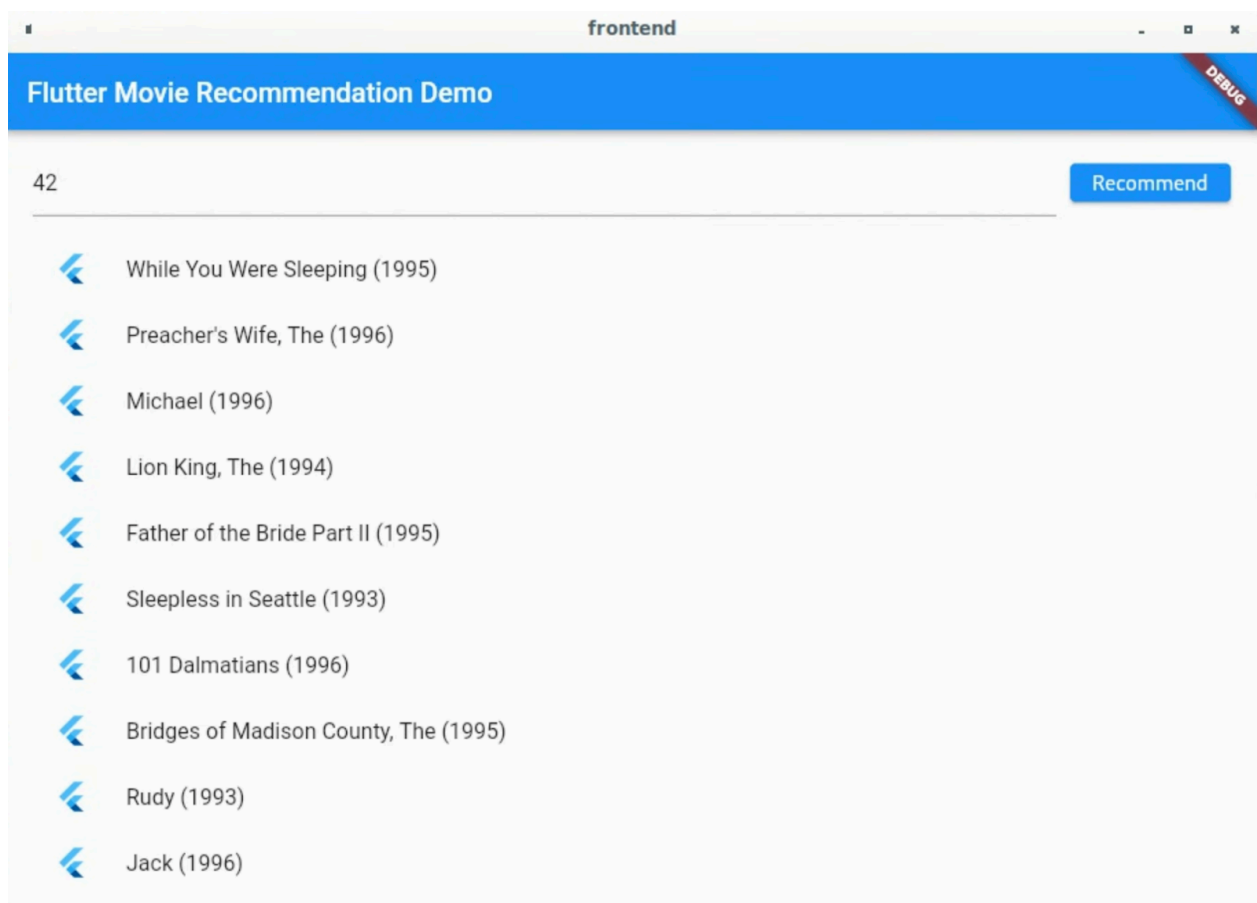
。

2. 按一下「Start debugging」圖示



，然後等待應用程式載入。

3. 輸入使用者 ID (即 42)，然後選取「建議」。



Mac

1. 如果是 Mac，由於應用程式會將 HTTP 要求傳送至後端，因此您需要設定適當的權利。詳情請參閱「[Entitlements and the App Sandbox](#)」。

將這段程式碼分別新增至 `step4/frontend/macOS/Runner/DebugProfile.entitlements` 和 `step4/frontend/macOS/Runner/Release.entitlements`：

```
<key>com.apple.security.network.client</key>
<true/>
```

2. 確認目標裝置在 VSCode 的狀態列中設為

macOS (darwin)

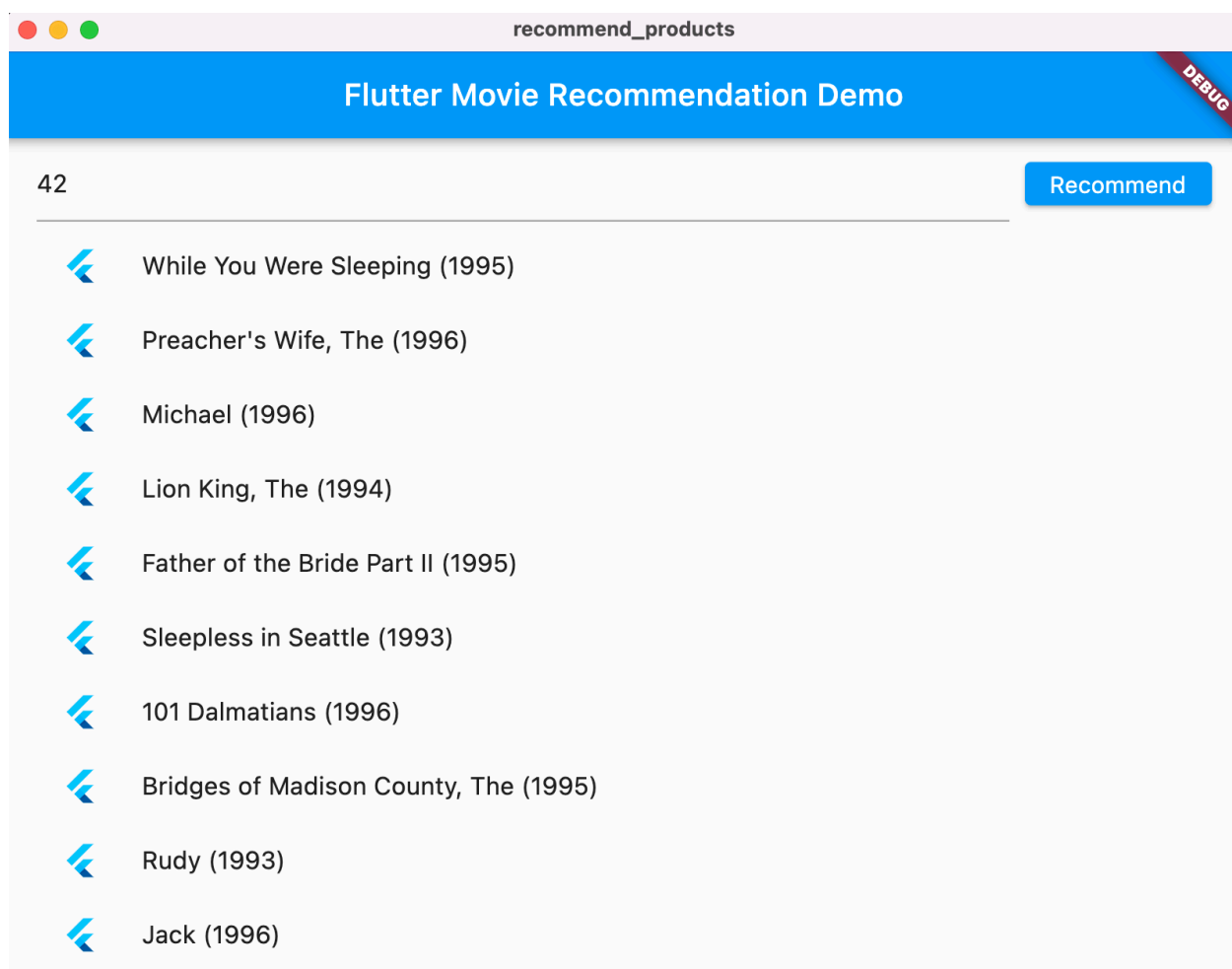
。

3. 按一下「Start debugging」圖示



，然後等待應用程式載入。

4. 輸入使用者 ID (即 42)，然後選取「建議」。



Windows

1. 確認目標裝置在 VSCode 的狀態列中設為

Windows (windows-x64)

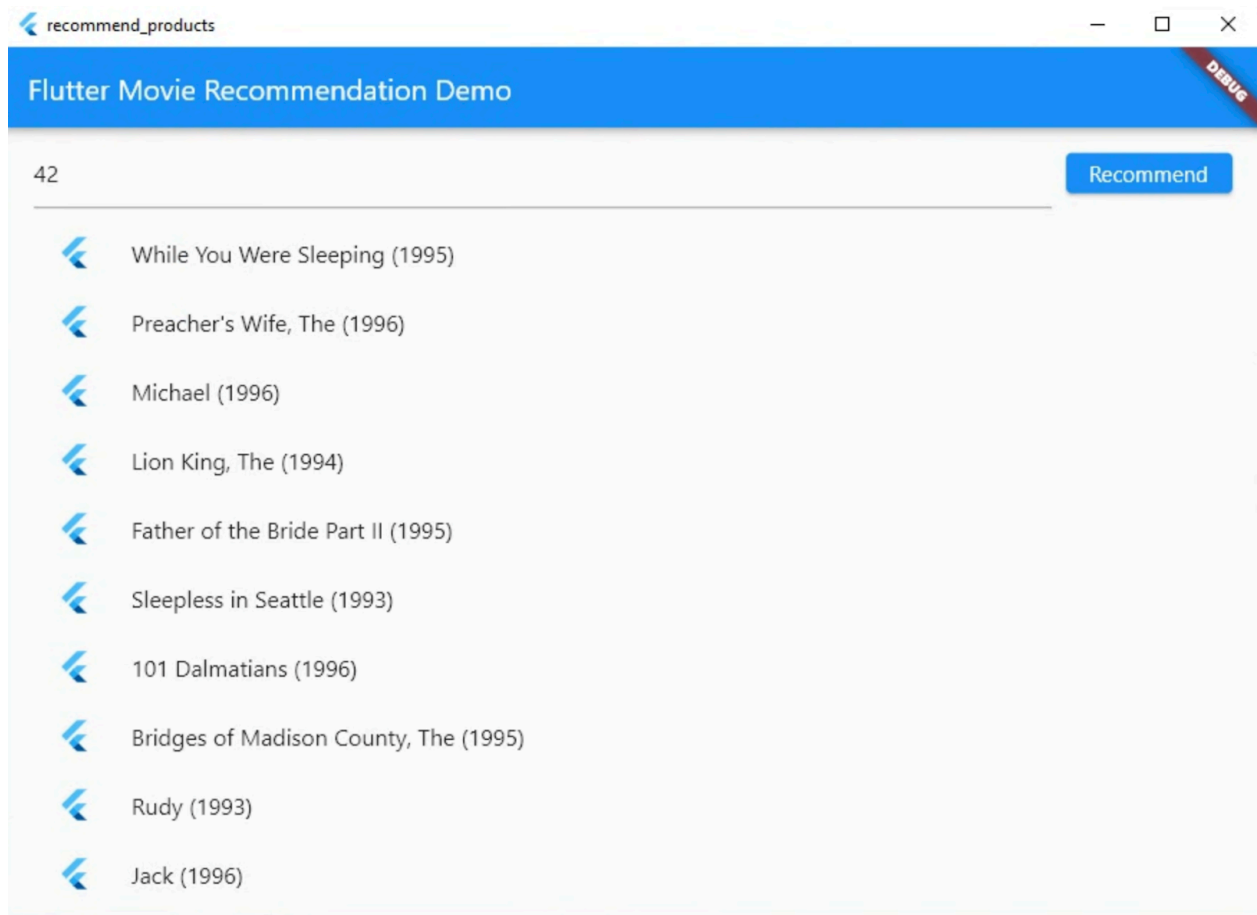
。

2. 按一下「Start debugging」圖示



，然後等待應用程式載入。

3. 輸入使用者 ID (即 42)，然後選取「建議」。



10. 步驟 5：在網頁平台上執行 Flutter 應用程式

您還可以為 Flutter 應用程式新增網頁支援功能。根據預設，Flutter 應用程式會自動啟用網頁平台，因此您只需要啟動即可。

1. 確認目標裝置在 VSCode 的狀態列中設為

Chrome (web-javascript)

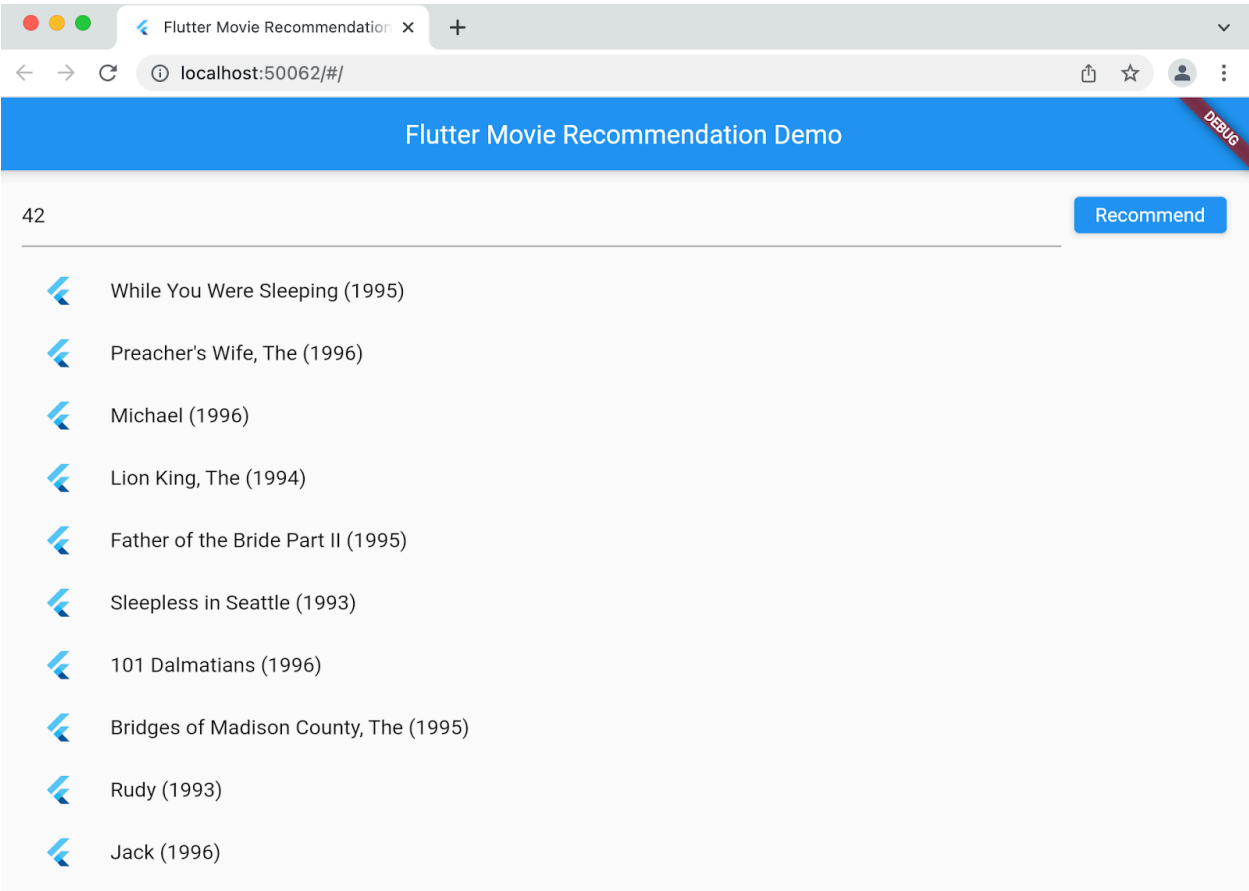
。

2. 按一下「開始偵錯」



，然後等待應用程式在 Chrome 瀏覽器中載入。

3. 輸入使用者 ID (即 42)，然後選取「建議」。



11. 恭喜

您已建構全端應用程式，可向使用者推薦電影！

雖然這個應用程式只會推薦電影，但您已瞭解如何建構強大的推薦引擎，並掌握在 Flutter 應用程式中使用推薦內容的技能。您可以輕鬆將所學內容套用至其他情境 (例如電子商務、美食和短片)。

瞭解詳情

- [TensorFlow Recommenders 首頁](#)
 - [TensorFlow Serving 首頁](#)
 - [Flutter 首頁](#)
 - [YouTube 上的「使用 TensorFlow 建構建議系統」系列影片](#)
-